

ArTE-VLA: Autoregressive Target Effects for Vision-Language-Action Models

Anonymous Authors

Abstract—Vision-language-action (VLA) models typically generate robot actions directly from visual observations and language instructions, leaving the intended physical evolution of the scene implicit. We introduce ArTE-VLA, a VLA that predicts how the language-referred target should change before generating an action. Given dual-view RGB observations and an instruction, ArTE-VLA infers an implicit target representation and autoregressively predicts target effects at four future horizons. Each effect describes translation, rotation, linear and angular velocity, and activity. The predicted effects are embedded as tokens and appended to the visual-language prefix, allowing the action expert to condition on a compact physical future. Simulator states provide effect supervision during training but are not required at inference. Across three evaluation seeds on all ten LIBERO-Object tasks, ArTE-VLA improves the success rate of a recipe-matched PI0.5 baseline from 70.9% to 84.3%, a 13.4 percentage-point gain over 1,500 paired episodes per method. Fixed-noise interventions further show that changing the predicted effect values produces consistent changes in generated actions. These results suggest that a compact, target-centric future representation can provide an effective and inspectable interface between VLA perception and control.

I. INTRODUCTION

Vision-language-action models have emerged as a promising route toward general robot policies [1], [2], [3], [4]. By combining pretrained visual-language representations with large collections of robot demonstrations, recent VLAs can follow open-vocabulary instructions and transfer across objects, tasks, and environments [5], [6], [1]. Most existing architectures, however, are optimized to map the current observation directly to an action sequence [7], [8]. The physical outcome that connects an instruction to an action—which object should move, in what way, and over what time scale—remains latent inside the policy.

This direct formulation creates two related difficulties. First, the action expert must simultaneously resolve language grounding, infer task-relevant dynamics, and generate precise controls [9], [10]. Second, the resulting computation is difficult to inspect: a successful rollout does not reveal whether the policy anticipated the intended object motion or relied on a less robust correlation [9], [11]. Predictive world models and visual chain-of-thought methods address this problem by forecasting future images or point trajectories before acting [12], [13], [14], [15]. Although expressive, these intermediate representations can be high-dimensional and need not isolate the physical state of the object that is central to the instruction [16].

We propose ArTE-VLA, which inserts a compact target-effect bottleneck into a pretrained VLA. Rather than predicting a complete future image, the model compresses its

visual-language prefix into an implicit representation of the primary target and predicts the target’s physical effects at four horizons. Each prediction contains a 3-D translation, a 3-D rotation, linear and angular velocity, and an activity logit. The predictions form an autoregressive chain: the effect at one horizon is used to predict the next. Horizon-specific hidden states and effect values are then projected into tokens that condition the flow-matching action expert.

To physically ground this bottleneck without changing the policy’s deployment inputs, we use simulator-derived target effects as privileged supervision during training only. We recover object states from the simulator states stored in LIBERO [17] demonstrations and construct aligned multi-horizon effect labels. These labels appear only in the auxiliary prediction loss; the action expert always receives model-predicted effects. At test time, ArTE-VLA uses the same RGB observations, instruction, and robot state as vanilla PI0.5 [5]. Consequently, structured physical supervision can shape the policy without introducing an object detector, simulator state, task identifier, or ground-truth target state into the deployed model. Figure 1 summarizes the resulting target-to-effect-to-action pathway and its separation between training supervision and deployment inputs.

Our main contributions are:

- We introduce a compact, target-centric physical-effect bottleneck for VLA policies. The bottleneck predicts a four-horizon autoregressive future and directly conditions the action expert.
- We develop a replay-based labeling pipeline that adds aligned target effects to robot demonstrations while preserving their original images, actions, states, and language instructions.
- We formulate context-preserving effect-value interventions that distinguish reliance on the overall effect-token pathway from sensitivity to explicit physical values and their temporal order.
- On LIBERO-Object [17], ArTE-VLA improves a matched PI0.5 baseline by 13.4 percentage points across three evaluation seeds and 1,500 paired episodes per method, while revealing substantial variation across target objects.

II. RELATED WORK

A. Vision-Language-Action Models

VLA models transfer pretrained vision-language priors to robot control through diverse manipulation trajectories [18]. Octo supports language or goal-image conditioning

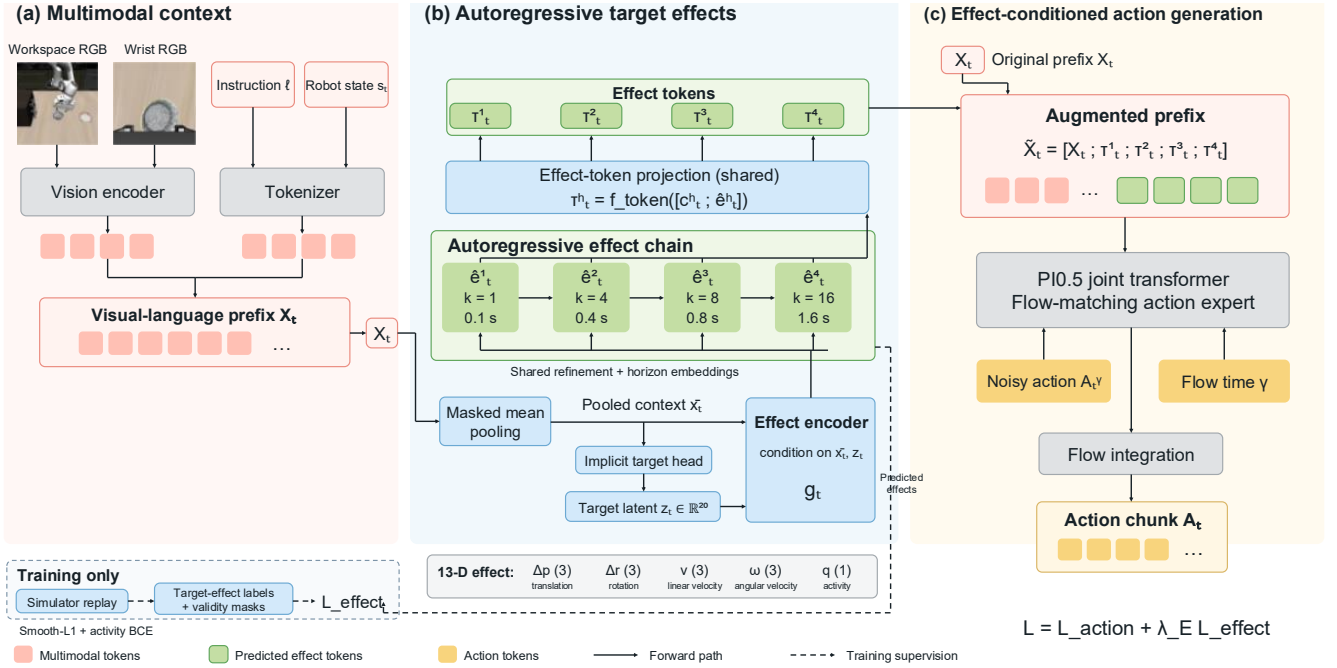


Fig. 1. Overview of ArTE-VLA. (a) Dual-view RGB observations, the instruction, and robot state form the original PI0.5 prefix X_t . (b) Masked pooling and an implicit target head seed a four-horizon autoregressive effect chain; each hidden state and predicted 13-D effect are projected into an effect token. (c) The effect tokens augment X_t and condition the flow-matching action expert, which generates the complete action chunk. The dashed path denotes simulator-derived effect labels and validity masks used only during training.

and adaptation to new observation and action spaces [19], while OpenVLA emphasizes an open, scalable VLA that can be fine-tuned for downstream manipulation [18]. Flow-matching policies avoid discretizing continuous controls: π_0 couples a vision-language backbone to a continuous action expert [20], and PI0.5 extends this family with heterogeneous co-training for open-world manipulation [5]. These systems primarily map the current multimodal context directly to actions. ArTE-VLA uses PI0.5 [5] as its backbone but adds a jointly supervised physical prediction interface that is consumed by the otherwise unchanged action expert.

B. Predictive Representations for Control

Predictive representations make intended scene evolution explicit before or alongside action generation [21]. GR-2 jointly learns video and action prediction from web videos and robot trajectories [22]; CoT-VLA generates future visual goals before actions [12]; and WorldVLA places image and action generation in a shared autoregressive formulation [13]. These visual futures retain appearance and multi-object context, but also encode scene content irrelevant to the commanded manipulation. [23], [24] ArTE-VLA neither synthesizes future observations nor performs explicit planning with learned dynamics. It predicts a low-dimensional physical change for one language-conditioned target and inserts it into the action context. Thus, visual prediction preserves rich scene evolution, whereas target effects expose compact values whose temporal order and influence on actions can be directly tested.

C. Object and Point Motion as Policy Guidance

Object-centered conditioning separates task-relevant entities from the scene [25], [26]. MOO extracts object cues from the image and command to support manipulation of unseen categories [27], but describes the current target rather than its desired evolution. Motion representations complement this grounding: Any-point Trajectory Modeling forecasts arbitrary image-point tracks for policy learning [14], and Track2Act converts predicted goal-conditioned tracks into object and end-effector motion plans [15]. ArTE-VLA instead predicts one instruction-grounded target’s multi-horizon 3-D translation, rotation, velocities, and activity, then injects these effects as tokens into an end-to-end VLA action expert. Because its target representation is implicit, it is not a standalone detector or calibrated tracker; the structured effects serve as a compact supervision and action-conditioning bottleneck.

III. METHOD

Figure 1 shows the complete computation. ArTE-VLA preserves the original PI0.5 visual-language prefix and adds a target-effect branch between prefix encoding and action generation. This branch infers a compact target latent, predicts four temporally ordered physical effects, and projects each prediction with its horizon-specific hidden state into a token appended to the original prefix. The action expert therefore retains the original multimodal context while receiving a structured prediction of the target’s future. During training, simulator replay supervises the effect predictions through an auxiliary loss; at deployment, every conditioning

effect is model-predicted and no simulator-derived quantity is required.

A. Problem Formulation

At time t , the policy receives workspace and wrist RGB observations $I_t = (I_t^{\text{ws}}, I_t^{\text{wr}})$, proprioceptive state s_t , and a language instruction ℓ . As in PI0.5, the normalized state is discretized and included in the language prompt, while both images are encoded into visual tokens. Let $X_t = \Phi(I_t, s_t, \ell)$ denote the resulting visual-language prefix. A conventional VLA directly models an action chunk $A_t = (a_t, \dots, a_{t+C-1})$ from X_t . We instead introduce an intermediate target-effect chain $E_t = (e_t^1, \dots, e_t^H)$ and use the factorization

$$p(A_t, E_t \mid I_t, s_t, \ell) = \left[\prod_{h=1}^H p_\phi(e_t^h \mid e_t^{<h}, X_t) \right] \times p_\theta(A_t \mid X_t, T(E_t)), \quad (1)$$

where $T(E_t)$ embeds the predicted effects into the PI0.5 prefix space. We use $H = 4$ temporal offsets $(k_1, \dots, k_H) = (1, 4, 8, 16)$, corresponding to $(0.1, 0.4, 0.8, 1.6)$ s at the 10-Hz control rate. The intermediate chain asks the model to first represent how the instructed target should evolve and then generate the complete action chunk conditioned on that evolution. This is a temporal autoregressive factorization, not a structural causal model.

B. Implicit Target Representation

The backbone produces $X_t \in \mathbb{R}^{N \times d}$, including visual patches from both cameras and tokens from the instruction-state prompt. Using a prefix validity mask m_i , we form a global context $\bar{x}_t$ and produce one compact target latent:

$$\bar{x}_t = \frac{\sum_{i=1}^N m_i X_{t,i}}{\sum_{i=1}^N m_i}, \quad z_t = f_{\text{target}}(\bar{x}_t) \in \mathbb{R}^{20}. \quad (2)$$

Here f_{target} is a layer-normalized two-layer MLP with a 256-D hidden layer. Our implementation retains the entity-slot dimension of the shared multi-entity predictor, but z_t occupies its sole valid slot and the remaining slots are masked. Joint conditioning on vision, language, and state allows z_t to encode the command-relevant object or articulated part without an explicit detector or object identifier. Importantly, z_t is not regressed to the current simulator state. Its semantics arise only through the downstream future-effect and action objectives, so we call it an *implicit target representation*, not a calibrated 20-D physical state.

C. Multi-Horizon Effect Chain

The effect chain first combines the target latent with the pooled backbone context,

$$g_t = \text{Enc}_E(f_z(z_t) + f_x(\bar{x}_t)). \quad (3)$$

All effect hidden states are 256-D; Enc_E and Refine_E are two-layer, four-head Transformer encoders. The latter is

shared across horizons and applied recursively. With $\hat{e}_t^0 = \mathbf{0}$ and a learned horizon embedding u_h ,

$$c_t^h = \text{Refine}_E(g_t + W_e \hat{e}_t^{h-1} + u_h), \quad (4)$$

$$\hat{e}_t^h = f_E(c_t^h). \quad (5)$$

Thus, a later prediction receives the model’s earlier prediction rather than a ground-truth previous effect. Each output has the fixed schema

$$\hat{e}_t^h = [\Delta p_t^h, \Delta r_t^h, v_t^h, \omega_t^h, q_t^h] \in \mathbb{R}^{13}, \quad (6)$$

where $\Delta p_t^h \in \mathbb{R}^3$ and $\Delta r_t^h \in \mathbb{R}^3$ describe translation and axis-angle rotation from t to $t + k_h$, $v_t^h, \omega_t^h \in \mathbb{R}^3$ are linear and angular velocities at $t + k_h$, and q_t^h is an activity logit. Translation and velocity are expressed in the end-effector frame at t . With R_t denoting the target orientation, the rotation increment is the logarithmic map of $R_t^\top R_{t+k_h}$. This target-centric effect schema separates intended physical change from the robot controls used to produce it.

D. Conditioning the Action Expert

The 13-D prediction captures explicit physical quantities, while its hidden state retains the horizon-specific context used to produce them. We therefore concatenate each prediction with its hidden state and project the result to the backbone width:

$$\tau_t^h = f_{\text{token}}([c_t^h; \hat{e}_t^h]) \in \mathbb{R}^d. \quad (7)$$

In our PI0.5 backbone, $d = 2048$. The four effect tokens are appended to the original prefix, $\tilde{X}_t = [X_t; \tau_t^1; \dots; \tau_t^4]$, and are visible to the flow-matching action expert. The original prefix is never replaced, so the expert retains direct access to the images, instruction, and state in addition to the predicted future. Moreover, supervision-validity masks are used only in the effect loss: all four predicted tokens remain present during action training and inference. This prevents episode-boundary information from leaking into the policy through a label-derived token mask.

Following PI0.5, let $\epsilon \sim \mathcal{N}(0, I)$ and $\gamma \in (0, 1)$ be the action-flow noise and sampled flow time. Training constructs $A_t^\gamma = (1 - \gamma)A_t + \gamma\epsilon$ with target velocity $u_t = \epsilon - A_t$. The action expert predicts $v_\theta(A_t^\gamma, \gamma \mid \tilde{X}_t)$ and minimizes

$$\mathcal{L}_{\text{action}} = \mathbb{E}_{A_t, \epsilon, \gamma} \left[\left\| v_\theta(A_t^\gamma, \gamma \mid \tilde{X}_t) - u_t \right\|_2^2 \right]. \quad (8)$$

At inference, numerical integration starts from Gaussian noise and follows the predicted vector field from $\gamma = 1$ to 0, yielding the complete action chunk rather than a residual correction to a separately generated action.

E. Training Objective

For each valid horizon, simulator replay supplies a continuous target $y_{t,h}^{1:12}$, activity label $y_{t,h}^{\text{act}}$, and validity mask $M_{t,h}$.

Let $K_t = \max(1, \sum_h M_{t,h})$. The two effect losses are

$$\mathcal{L}_{\text{reg}} = \mathbb{E}_t \left[\frac{1}{K_t} \sum_h M_{t,h} \frac{1}{12} \sum_{j=1}^{12} \rho(\hat{e}_{t,h}^j - y_{t,h}^j) \right], \quad (9)$$

$$\mathcal{L}_{\text{act}} = \mathbb{E}_t \left[\frac{1}{K_t} \sum_h M_{t,h} \text{BCEWithLogits}(q_t^h, y_{t,h}^{\text{act}}) \right], \quad (10)$$

where ρ is Smooth- L_1 . The full end-to-end objective is

$$\mathcal{L} = \mathcal{L}_{\text{action}} + \lambda_E (\mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{act}}), \quad (11)$$

with $\lambda_E = 1$. The reported model applies no component- or class-specific reweighting: all 12 continuous dimensions are weighted equally, including for inactive targets, and positive and negative activity labels receive equal weight before the valid-horizon normalization above. All three loss terms are optimized jointly. Crucially, there is no teacher forcing in the effect recursion, and the action expert is always conditioned on predicted effects rather than replay labels. Training and deployment therefore share the same target-to-effect-to-action path.

F. Replay-Based Effect Supervision

We obtain structured supervision without changing the policy’s deployment inputs. For every demonstration frame, the enrichment procedure restores the stored flattened MuJoCo state and calls the simulator’s kinematic forward pass; it never steps the recorded action. Let p_t , R_t , v_t , and ω_t denote the target’s position, orientation, linear velocity, and angular velocity in the restored state, and let $R_{\text{ee},t}$ denote the current end-effector orientation. Matching stored states at t and $t + k_h$ yields

$$\Delta p_t^h = R_{\text{ee},t}^\top (p_{t+k_h} - p_t), \quad (12)$$

$$\Delta r_t^h = \text{Log}(R_t^\top R_{t+k_h}), \quad (13)$$

$$v_t^h = R_{\text{ee},t}^\top v_{t+k_h}, \quad (14)$$

$$\omega_t^h = R_{\text{ee},t}^\top \omega_{t+k_h}. \quad (15)$$

The target is marked active if $\|\Delta p_t^h\|_2$ exceeds 1 mm, $\|\Delta r_t^h\|_2$ exceeds 1° , $\|v_t^h\|_2$ exceeds 0.01 m/s, or $\|\omega_t^h\|_2$ exceeds 0.05 rad/s. A horizon is valid only when the same target exists at both frames and $t + k_h$ remains inside the episode.

Object names and simulator body identifiers are retained only as enrichment provenance. The enriched data preserve the original RGB observations, proprioception, actions, instructions, and frame indices. During training, the task index identifies the teacher entity in the multi-entity replay tensor. The selected labels are then packed into model slot 0, the sole active target slot for all ten LIBERO-Object tasks. Neither the task index nor the teacher slot, object identity, effect label, or simulator state is embedded by the policy. At deployment, the model uses only the same images, instruction, and robot state as the PI0.5 baseline.

G. Effect Interventions

To test what information reaches the action expert, we intervene after effect prediction while fixing the observation, language, checkpoint, initial action noise, and flow-integration schedule. *Mask* hides all effect tokens from the action expert, measuring dependence on the added pathway as a whole. *Zero-12D* sets only the 12 continuous values in Eq. (6) to zero while preserving each horizon context c_t^h and activity logit. *Reverse* assigns the four 13-D predictions in reverse temporal order, and *Shuffle* substitutes predictions from another sample. For these three effect-value interventions, the altered effects are re-encoded with the receiving sample’s original hidden contexts; its visual, language, and current-state information is never swapped. The resulting tests distinguish dependence on the overall token pathway from sensitivity to explicit physical values and their temporal organization.

IV. EXPERIMENTS

A. Experimental Setup

1) *Benchmark and Data*: We evaluate on LIBERO-Object, which contains ten language-conditioned tabletop manipulation tasks [17]. The training set contains 454 available demonstration episodes and 66,984 frames. Replay enrichment yields 253,423 valid target-horizon labels, of which 170,107 are active. We verify frame-level alignment between the original and enriched datasets and observe a maximum action-alignment error of 2.90×10^{-8} . Terminal horizons without a valid successor state are masked. We additionally evaluate the complete five-task atomic PickPlace family in RoboCasa [28]: Counter to Cabinet, Counter to Stove, Drawer to Counter, Sink to Counter, and Toaster to Counter.

2) *Training Setup*: Both vanilla PI0.5 and ArTE-VLA are initialized from `lerobot/pi05_base` and jointly fine-tune the complete model; we use neither parameter-efficient adaptation nor a frozen vision encoder. Within each benchmark, ArTE-VLA differs from vanilla PI0.5 only by the target head, effect chain, effect-token projection, and the auxiliary loss in Eq. (11). The benchmark-specific settings are summarized below.

Benchmark-specific training configurations		
Setting	LIBERO-Object	RoboCasa
Updates	30,000	15,000
GPUs	8	8
Global batch size	64	128
Per-GPU batch size	8	16
Camera views	2	3
Image size (stored/model)	256/224	256/224
Warmup/decay steps	1,000/30,000	500/15,000

All runs use AdamW with peak learning rate 2.5×10^{-5} , weight decay 0.01, $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, and global gradient-norm clipping at 1.0. After the tabulated linear warmup, the learning rate follows cosine decay to 2.5×10^{-6} . All runs use bfloat16 precision, gradient checkpointing, seed 1000, and no random image augmentation. Square images are resized with aspect-preserving padding; LIBERO-Object

uses workspace and wrist views, whereas RoboCasa uses two agent views and one eye-in-hand view.

We evaluate the final checkpoint without validation-based model selection or online environment evaluation during training. Vanilla PI0.5 has 4.143B trainable parameters; ArTE-VLA has 4.149B, adding 5.381M parameters (0.13%). Steady-state checkpoint intervals indicate approximate eight-H2O training times of 12.5 hours for the LIBERO-Object vanilla model and 14.1 hours for ArTE-VLA.

3) *Evaluation Protocol*: For each task, we evaluate both policies from the same 50 demonstration-derived initial simulator states under evaluation seeds 1000, 2000, and 3000. This produces 500 paired episodes per method and seed, or 1,500 episodes per method overall. All three evaluations reuse the same seed-1000 training checkpoints; they measure stochastic rollout variability rather than variability across independent training runs. Rollouts use synchronous batches of ten environments, ten flow-matching inference steps, and a fixed time horizon. Success is recorded without terminating the episode early. We report per-task and aggregate success, paired rescue and regression counts, and the exact two-sided McNemar test. Because these resets reconstruct expert demonstration states rather than the benchmark’s standard initial-state set, we keep this protocol separate from comparisons to previously published LIBERO results.

For RoboCasa, each task is evaluated on 20 task-seeded episodes under evaluation seed groups 1000, 2000, and 3000. This gives 100 episodes per method and seed group, or 300 episodes per method overall. The two methods use identical reset seeds and environment-provided instructions.

4) *Baseline and Ablation*: The primary baseline is recipe-matched vanilla PI0.5. The only separately trained ablation varies the auxiliary effect-loss weight over $\lambda_E \in \{0.1, 0.3, 1.0\}$ while holding the remaining recipe fixed. We also apply removal, zero-value, reverse-order, and cross-sample shuffle interventions to the same ArTE-VLA checkpoint; these are inference-time sensitivity tests, not separately trained ablations. The weight sweep tests optimization sensitivity, while the interventions test whether the trained action expert responds to the effect-token pathway and its explicit values. Because we do not include capacity-matched or structural controls, these experiments do not isolate the contributions of added parameters, autoregression, or hidden effect context.

5) *Metrics*: Closed-loop success is the primary metric. We report per-task and aggregate success, together with the absolute percentage-point difference from vanilla PI0.5. For the paired LIBERO-Object rollouts, we additionally report rescue and regression counts and an exact two-sided McNemar test. Action sensitivity is measured by the fraction of action chunks with any numerical change, the maximum absolute elementwise difference, and the relative root-mean-square (RMS) distance; the latter two are summarized by their medians across observations. Trainable-parameter overhead is reported in the training setup.

B. Closed-Loop Performance

Table I compares ArTE-VLA with recipe-matched vanilla PI0.5 on LIBERO-Object and five RoboCasa pick-and-place tasks. Across the three evaluation seeds, target-effect conditioning improves mean success from 70.9% to 84.3%, an absolute gain of 13.4 percentage points. The gains for seeds 1000, 2000, and 3000 are 12.8, 14.8, and 12.6 points, respectively. Pooling the 1,500 paired rollouts, ArTE-VLA produces 380 rescues and 179 regressions. The corresponding exact two-sided McNemar test yields $p = 1.20 \times 10^{-17}$.

The aggregate improvement is dominated by two large rescues. On Butter and Chocolate Pudding, vanilla PI0.5 achieves only 4.7% and 14.7%, whereas ArTE-VLA reaches 90.0% and 86.7%, respectively. The gain is therefore not a uniform shift across tasks. Ketchup and Milk decrease by 33.3 and 27.3 points, respectively, and regress under every evaluation seed. BBQ Sauce decreases by 5.3 points but includes one tied seed. Salad Dressing also changes sign across seeds. This heterogeneity indicates that target effects can resolve severe failures of a direct policy, but can also introduce task-dependent interference.

C. Effect-Loss Ablation and Action Sensitivity

a) *Effect-loss weight*: Table II reports seed-1000 sensitivity to λ_E ; the training recipe is otherwise matched within each benchmark. Additional evaluation seed groups are not complete for every weight, so we restrict this comparison to the shared seed-1000 setting.

b) *Action sensitivity*: We perform a fixed-noise action audit on 32 observations drawn equally from Butter and Chocolate Pudding. Repeating the unmodified forward computation gives zero numerical action difference. Each value intervention changes the action chunk for all 32 samples. Zeroing the 12 continuous effect values yields a median maximum absolute action difference of 0.00956 and a median relative RMS distance of 0.943%. Reversing the horizon order increases these medians to 0.01198 and 1.109%, while cross-sample shuffling yields 0.01277 and 1.306%.

These results establish computational dependence of the action expert on the predicted physical values. They do not establish that those values alone account for the closed-loop improvement: each effect token also contains a learned horizon context, and the zero-value median lies slightly below the prespecified 1% relative-RMS decision threshold. Closed-loop interventions over the full task distribution are therefore necessary to quantify the independent control value of the structured prediction.

TABLE I

SUCCESS (%) ON LIBERO-OBJECT AND ROBOCASA PICKPLACE-5 ACROSS THREE EVALUATION SEED GROUPS USING FIXED SEED-1000 TRAINING CHECKPOINTS. ENTRIES ARE MEANS ACROSS SEEDS. EACH LIBERO-OBJECT SEED CONTAINS 50 PAIRED EPISODES PER TASK; EACH ROBOCASA SEED CONTAINS 20.

Benchmark	Task	PI0.5	ArTE-VLA	Δ vs. PI0.5
LIBERO-Object	Alphabet Soup	77.3	74.7	-2.7
	Cream Cheese	74.0	96.0	+22.0
	Salad Dressing	86.7	87.3	+0.7
	BBQ Sauce	93.3	88.0	-5.3
	Ketchup	97.3	64.0	-33.3
	Tomato Sauce	70.7	90.0	+19.3
	Butter	4.7	90.0	+85.3
	Milk	95.3	68.0	-27.3
	Chocolate Pudding	14.7	86.7	+72.0
	Orange Juice	95.3	98.7	+3.3
	Overall	70.9	84.3	+13.4
RoboCasa	Counter to Cabinet	41.7	25.0	-16.7
	Counter to Stove	76.7	28.3	-48.3
	Drawer to Counter	40.0	16.7	-23.3
	Sink to Counter	70.0	45.0	-25.0
	Toaster to Counter	88.3	73.3	-15.0
	Overall	63.3	37.7	-25.7

TABLE II

SEED-1000 EFFECT-LOSS-WEIGHT ABLATION. LIBERO EVALUATION
BATCH SIZE IS 20 FOR $\lambda_E = 0.3$ AND 10 OTHERWISE.

λ_E	LIBERO-Object		RoboCasa	
	Success	Δ	Success	Δ
0.1	78.4	+7.6	50.0	-10.0
0.3	74.8	+4.0	44.0	-16.0
1.0	83.6	+12.8	41.0	-19.0

TABLE III

FIXED-NOISE EFFECT INTERVENTIONS ON 32 OBSERVATIONS. ACTION METRICS ARE MEASURED RELATIVE TO THE UNMODIFIED COMPUTATION.

Intervention	Intervention content	Max abs. median	Rel. RMS median (%)	Changed (%)
Repeat	Identical-condition repeat	0	0	0
Zero-12D	Zero 12-D future-state values	0.00956	0.943	100
Reverse	Reverse values across horizons	0.01198	1.109	100
Shuffle	Cross-sample substitution	0.01277	1.306	100

D. Discussion and Limitations

The task-level results show that the aggregate LIBERO-Object gain is not a uniform improvement. The large rescues on Butter and Chocolate Pudding are consistent with target-effect supervision providing an additional object-motion signal when the direct policy fails. By contrast, the regressions on Ketchup, Milk, and BBQ Sauce could reflect inaccurate target effects, entanglement of object and task context in the implicit target representation, or competition between effect and visual-language tokens inside the action expert. These explanations remain hypotheses: distinguishing them requires comparing effect error and action sensitivity on

matched rescue and regression episodes.

The experimental evidence has several further limits. All reported models use one training seed; Table I varies only rollout seeds and uses an expert-demonstration reset protocol, while the weight comparison in Table II is restricted to the shared seed-1000 evaluation. The current experiments also lack capacity-matched, value-free, and structural controls. Consequently, they do not isolate the contribution of explicit physical values from added capacity, autoregressive prediction, or the learned horizon context carried by each effect token. Likewise, the 32-observation audit establishes computational dependence rather than an independent closed-loop benefit of the predicted values.

Finally, effect supervision relies on privileged object states available in simulation or instrumented datasets. Extending the method to unconstrained real-world data would require reliable state estimation or pseudo-labeling. Independent training-seed replication, standard-initialization evaluation, completed cross-benchmark weight sweeps, matched controls, and closed-loop interventions over the full task distribution are needed to establish robustness, causal attribution, and generalization.

V. CONCLUSION

We presented ArTE-VLA, a VLA architecture that inserts a structured physical prediction pathway between visual-language understanding and continuous action generation. From the multimodal context, the model forms an implicit representation of the language-referred target, autoregressively predicts a compact chain of target effects at four future horizons, and conditions the action expert on the resulting tokens. Privileged object states are used only to construct training targets, so inference retains the original RGB-

language interface without requiring an external detector or state estimator.

On LIBERO-Object, ArTE-VLA improves aggregate success from 70.9% to 84.3% over three evaluation seeds. Across 1,500 paired rollouts, it produces 380 rescues and 179 regressions, with a significant exact McNemar test. In the shared seed-1000 setting, all three tested effect-loss weights outperform the recipe-matched baseline on LIBERO-Object. Fixed-noise interventions further show that perturbing effect values or their temporal order changes the generated action chunks consistently. This establishes that the action expert uses the effect pathway computationally, although the current audit does not isolate an independent closed-loop benefit of the explicit physical values.

The evidence therefore supports a focused conclusion: structured, autoregressive target effects can improve aggregate control performance while providing an inspectable intermediate representation for action generation. It does not yet establish uniform gains across tasks, robustness across training initializations, or causal attribution to the continuous effect values alone. Addressing these questions with matched structural controls, independent training seeds, standard benchmark initializations, and full-distribution closed-loop interventions is an important next step toward determining when effect-conditioned VLAs generalize reliably beyond the present setting.

REFERENCES

- [1] K. Kawaharazuka, J. Oh, J. Yamada, I. Posner, and Y. Zhu, “Vision-language-action models for robotics: A review towards real-world applications,” *IEEE Access*, 2025.
- [2] X. Li, P. Li, M. Liu, D. Wang, J. Liu, B. Kang, X. Ma, T. Kong, H. Zhang, and H. Liu, “Towards generalist robot policies: What matters in building vision-language-action models,” *arXiv preprint arXiv:2412.14058*, vol. 3, no. 7, p. 8, 2024.
- [3] Y. Ma, Z. Song, Y. Zhuang, J. Hao, and I. King, “A survey on vision-language-action models for embodied ai,” *IEEE Transactions on Neural Networks and Learning Systems*, 2026.
- [4] R. Shao, W. Li, L. Zhang, R. Zhang, Z. Liu, R. Chen, and L. Nie, “Large vlm-based vision-language-action models for robotic manipulation: A survey,” *arXiv preprint arXiv:2508.13073*, 2025.
- [5] Physical Intelligence *et al.*, “ $\pi_0.5$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [6] M. U. Din, W. Akram, L. S. Saoud, J. Rosell, and I. Hussain, “Vision language action models in robotic manipulation: A systematic review,” *arXiv preprint arXiv:2507.10672*, 2025.
- [7] R. Sapkota, Y. Cao, K. I. Roumeliotis, and M. Karkee, “Vision-language-action (vla) models: Concepts, progress, applications and challenges,” *arXiv preprint arXiv:2505.04769*, 2025.
- [8] Y. Zhong, F. Bai, S. Cai, X. Huang, Z. Chen, X. Zhang, Y. Wang, S. Guo, T. Guan, K. N. Lui *et al.*, “A survey on vision-language-action models: An action tokenization perspective,” *arXiv preprint arXiv:2507.01925*, 2025.
- [9] S. Wang, J. Shi, Z. Fu, X. He, F. Liu, C. Yang, Y. Zhou, Z. Fei, J. Gong, J. Fu *et al.*, “World action models: The next frontier in embodied ai,” *arXiv preprint arXiv:2605.12090*, 2026.
- [10] W. Guo, Y. Wang, R. Zhou, Y. Zhang, R. Fang, Y. Pang, W. Xu, and G. Huang, “Action with visual primitives,” *arXiv preprint arXiv:2605.22183*, 2026.
- [11] H. Xu, S. Zheng, H. Luo, W. Zhang, Z. Xi, and Z. Lu, “Unmasking the illusion of embodied reasoning in vision-language-action models,” *arXiv preprint arXiv:2604.18000*, 2026.
- [12] Q. Zhao *et al.*, “CoT-VLA: Visual chain-of-thought reasoning for vision-language-action models,” *arXiv preprint arXiv:2503.22020*, 2025.
- [13] J. Cen *et al.*, “WorldVLA: Towards autoregressive action world model,” *arXiv preprint arXiv:2506.21539*, 2025.
- [14] C. Wen, X. Lin, J. So, K. Chen, Q. Dou, Y. Gao, and P. Abbeel, “Any-point trajectory modeling for policy learning,” *arXiv preprint arXiv:2401.00025*, 2024.
- [15] H. Bharadhwaj, R. Mottaghi, A. Gupta, and S. Tulsiani, “Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation,” *arXiv preprint arXiv:2405.01527*, 2024.
- [16] L. Zhong, Y. Liu, Y. Wei, Z. Xiong, S. Liu, and G. Ren, “Acot-vla: Action chain-of-thought for vision-language-action models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 8152–8162.
- [17] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [18] M. J. Kim *et al.*, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [19] Octo Model Team *et al.*, “Octo: An open-source generalist robot policy,” *arXiv preprint arXiv:2405.12213*, 2024.
- [20] K. Black *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [21] J. Ding, Y. Zhang, Y. Shang, Y. Zhang, Z. Zong, J. Feng, Y. Yuan, H. Su, N. Li, N. Sukiennik *et al.*, “Understanding world or predicting future? a comprehensive survey of world models,” *ACM Computing Surveys*, vol. 58, no. 3, pp. 1–38, 2025.
- [22] C.-L. Cheang *et al.*, “GR-2: A generative video-language-action model with web-scale knowledge for robot manipulation,” *arXiv preprint arXiv:2410.06158*, 2024.
- [23] X. Xu, H. Li, J. Ye, Y. Chen, J. Zeng, X. Chen, L. Xu, D. Lin, W. Li, and J. Pang, “Futurevla: Joint visuomotor prediction for vision-language-action model,” *arXiv preprint arXiv:2603.10712*, 2026.
- [24] C. Finn and S. Levine, “Deep visual foresight for planning robot motion,” in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 2786–2793.
- [25] P. R. Kasu and D. Um, “Object-centric diffusion policies for real-world robotic-arm imitation learning,” *Frontiers in Robotics and AI*, vol. 13, p. 1829436, 2026.
- [26] K. Wang, K. Fan, Y. Fu, S. Lin, H. Luo, D. Seita, Y. Fu, Y.-G. Jiang, and X. Xue, “Ocr: Object-centric learning with 3d and tactile priors for human-to-robot action transfer,” *arXiv preprint arXiv:2603.14401*, 2026.
- [27] A. Stone *et al.*, “Open-world object manipulation using pre-trained vision-language models,” *arXiv preprint arXiv:2303.00905*, 2023.
- [28] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu, “Robocasa: Large-scale simulation of everyday tasks for generalist robots,” in *Robotics: Science and Systems (RSS)*, 2024.